

Cordell Programming Language (CPL): A Lightweight Systems-Language Testbed

Nikolai Fot

Alexander Vinarskiy

Compiler Technology Department,

Ivannikov Institute for System Programming of the Russian Academy of Sciences (ISP RAS)

May 25, 2026

Abstract

Cordell Programming Language (CPL) is a deliberately restricted systems programming language and compiler infrastructure designed for low-level experimentation. Its syntax is close to C and assembly in intent, but it does not implement a C subset. The language exposes explicit control over primitive values, pointers, arrays, strings, lightweight containers, entry points, system calls, inline assembly, and system-oriented annotations, while intentionally omitting classes, enums, garbage collection, memory-safety guarantees, and production-language ecosystem assumptions. The compiler is organized as a multi-stage pipeline from preprocessing and tokenization to AST construction, semantic checking, HIR generation, SSA construction, high-level optimization, LIR lowering, instruction selection, register allocation, assembly emission, and linking. This report documents the current language core, compiler pipeline, implemented diagnostics, optimization passes, testing infrastructure, target status, preliminary microbenchmarks, limitations, and related work. The goal is not to present CPL as a verified or production-ready replacement for established systems languages such as C or Rust, but as a compact experimental platform for studying compiler architecture, systems-language design, optimization, and diagnostics.

1 Introduction

Systems programming languages expose machine-level details while attempting to keep programs tractable. Mature languages such as C [15, 8], C++ [17, 18], Rust [9, 7], and Zig [21] provide large ecosystems, but their full syntactic and semantic complexity makes them expensive targets for compiler experiments. Conversely, many teaching compilers are intentionally small but do not expose enough low-level mechanisms to study system-oriented code generation, platform-specific interfaces, and optimization under realistic constraints.

CPL is not a C subset. A faithful C-like subset would either require a substantially more complex parser and semantic model, or would create misleading expectations that the rest of C should also be supported. The current design instead favors a compact parser-friendly

grammar, explicit syntax for functions and pointer-like operations, and compiler-supported low-level constructs.

The contribution of this report is a consolidated technical description of the current Cordell compiler implementation:

- a concise language overview and design rationale;
- the current multi-stage compiler pipeline;
- the HIR, LIR, selected LIR, and assembly form of a small example;
- the implemented static-analysis split between AST and IR diagnostics, including Z3-backed reasoning for selected conditions;
- the optimization passes currently documented for the compiler;
- preliminary microbenchmark results and their limitations;
- a related-work discussion positioning CPL among systems-language, educational-compiler, and verified-compiler projects.

2 Design Goals and Non-Goals

2.1 Goals

CPL is designed around five goals.

Low-level explicitness. The language is intended to stay close to assembly and C. It exposes pointers, explicit dereferencing, C-like containers, direct system calls, inline assembly, manual entry points, and annotations for low-level placement and control.

Small and readable core. The language deliberately avoids a large surface syntax. Instead of emulating all of C, CPL provides a compact grammar and a small set of orthogonal features. This keeps parsing and lowering simple enough for compiler experimentation.

Compiler experimentation. The compiler is a testbed for SSA construction [5], static diagnostics, constant propagation, constant folding, common subexpression elimination, loop-invariant code motion, peephole transformations, branch layout annotations, tail-recursion elimination, function-inlining heuristics, compact neural-model-guided inlining experiments, and register-allocation-sensitive low-level optimization.

System-oriented portability. CPL tries to minimize dependencies on a host standard library. The long-term intent is that the compiler and its runtime support can be ported to small or custom operating systems by providing basic I/O and platform APIs.

Educational value. The implementation is meant to expose a full compiler pipeline while staying small enough to inspect. This makes it useful for studying how language syntax, intermediate representations, optimizations, and backend constraints interact.

2.2 Non-Goals

CPL does not claim any of the following properties:

- formal verification of the compiler;
- memory safety or ownership checking;
- production readiness;
- source-level compatibility with C;
- a general-purpose standard library or package ecosystem.

CPL supports lightweight containers as C-like aggregates, but deliberately avoids classes, constructors, destructors, inheritance, traits, and ownership-based object models. This keeps the language focused on primitive values, pointers, arrays, containers, functions, and explicit low-level constructs.

3 Language Overview

3.1 Program Entry

A CPL program can use a **start** block as its static entry point. The entry block does not return a value with **return**; it must terminate through **exit**. An ordinary function can also become an entry point through the **@[entry]** annotation.

```
start() {
    exit 0;
}
```

Listing 1: Minimal CPL entry point.

The entry point may accept arguments, such as **argc** and **argv**, and can be annotated for platform-specific entry behavior. The **naked** annotation disables default entry and exit routines, which is useful for system-level code that must manage its own prologue and epilogue. On i386 this also affects how stack arguments are addressed, as discussed in Section 4.4.

3.2 Types

CPL uses permissive static typing. Variables do not dynamically change type, widening conversions may be inserted implicitly, and narrowing conversions require an explicit **as cast**. The primitive integer and floating-point types include **i8**, **u8**, **i16**, **u16**, **i32**, **u32**, **i64**, **u64**, **f32**, **f64**, and the void-like function return type **i0**. Boolean-like logic follows the C convention: zero is false and non-zero is true.

3.3 Pointers, Strings, and Arrays

The language exposes pointer types through the **ptr** modifier. The **ref** keyword obtains a pointer to an object or string literal, and **dref** reads or writes through a pointer.

```
i32 x = 123;
ptr i32 p = ref x;
i32 y = dref p;
dref p = y + 1;
```

Listing 2: Pointer operations in CPL.

Strings and arrays are distinct built-in containers. A **arr[0, i8]** allocates a terminated byte sequence, while **arr** declares a fixed-size array. String literals placed independently in code reside in a read-only section-like area and must be referenced explicitly.

```
arr msg[0, i8] = "Hello world!";
arr xs[4, i32] = { 1, 2, 3, 4 };
ptr i8 p = ref "Hello, World!\n";
```

Listing 3: Strings and arrays.

3.4 Containers

CPL containers are lightweight C-like aggregate types. They group fields of different types in one named layout and are intended for explicit systems code rather than for object-oriented programming.

```
container pair {
    i32 first;
    i32 second;
}
```

Listing 4: A simple CPL container.

By default, container fields use the target architecture’s default maximum alignment policy. A container can override this with the **align** annotation. For example, **@[align(1)]** requests a packed layout, which is useful when the programmer needs byte-exact representation.

```
@[align(1)]
container packed_value {
    i8 a;
    i16 b;
    i32 c;
} :/ packed, occupies 7 bytes on the stack /:
```

Listing 5: Packed container layout.

In addition to fields, a container may declare functions. These functions are ordinary CPL functions attached syntactically to the container definition. They do not make the container a class, and they do not introduce implicit constructors or destructors.

```
container counter {
    function default_value() -> i32 {
        return 100;
    }
}

counter c;
c.default_value();
```

Listing 6: Container function.

The `self` annotation marks a container function whose first explicit argument is the receiver pointer. Calls to such functions may use method-like syntax, and the parser can pass the container pointer automatically. This is syntax sugar only: a `self` function must still declare its receiver argument explicitly.

```
container counter {
    i32 val;

    @[self]
    function init(ptr counter self) -> i0 {
        self.val = 0;
    }
}

counter c;
c.init();
c.val; :/ 0 /:
```

Listing 7: A self container function.

Containers should therefore be treated as modified C structures with optional attached functions. Initialization, cleanup, and ownership-like behavior must be written explicitly by the programmer.

3.5 Control Flow

CPL provides `if`, `while`, `loop`, and `switch`. A semicolon separates the condition from the body in conditional constructs. The `loop` construct represents an infinite loop unless terminated with `break` or annotated with `@[counter(n)]`. The `switch` construct supports fallthrough by default, while `@[no_fall]` inserts break-like behavior into cases.

```
if x > 0; {
    x -= 1;
}
else {
    x = 0;
}

@[counter(10)] loop {
    x += 1;
}
```

Listing 8: Control flow.

3.6 Functions

Functions are declared with the `function` keyword. CPL supports prototypes, default arguments, local functions, function overloading with restrictions, generic

functions, function pointers, lambdas without closure capture, and variadic arguments. Global functions use `glob` when their symbol name must be preserved for external linkage.

```
function add(i32 a, i32 b = 1) -> i32 {
    return a + b;
}

glob function exported(i32 x) -> i32;
```

Listing 9: Function forms.

Function pointers have no signature-level static information. This keeps them flexible but limits static checking, overload resolution, and default-argument insertion after a function is stored as a raw pointer.

3.7 System-Level Facilities

CPL provides two built-in low-level mechanisms: `syscall` and `asm`. A `syscall` expression is target-dependent and accepts a platform-specific argument list. Inline assembly supports argument substitution through numbered placeholders.

```
i32 a = 0;
i32 ret;
asm(a, ret) {
    "push rax",
    "mov rax, %0",
    "syscall",
    "mov %1, rax",
    "pop rax"
}
```

Listing 10: Inline assembly with placeholders.

Inline assembly is intentionally powerful and fragile. It is copied close to directly into the generated assembly after placeholder substitution, so the programmer must preserve registers and match the selected target syntax. The compiler does not optimize inside inline assembly blocks.

3.8 Annotations

Annotations extend the small grammar with low-level behavior. Table 1 summarizes the system-oriented annotations currently available in the implementation.

These annotations are not intended to make CPL a high-level language. Their role is to expose system-level control without turning the language into a high-level object-oriented system.

3.9 Inlining Annotations

CPL provides an `inline` annotation to help the optimizer decide whether a function should be expanded at its call sites. The annotation is intentionally close in spirit to the similarly named Rust annotation: it does not change the source-level meaning of the function, but it communicates an optimization preference to the compiler.

```
@[inline]
function foo(); :/ Function will be inlined with
higher odds /:
```

Table 1: Selected CPL annotations.

Annotation	Purpose	Example
<code>naked</code>	Disable default entry and exit routines.	<code>@[naked] start()</code>
<code>align</code>	Align a local or global object.	<code>@[align(16)] glob i32 a;</code>
<code>section</code>	Place code or data in a target section.	<code>@[section(".data")]</code>
<code>nosection</code>	Emit a declaration without an explicit section directive.	<code>@[nosection] glob function f()</code>
<code>address</code>	Place a function at a specific address.	<code>@[address(0x1000)]</code>
<code>entry</code>	Mark a function as an entry point.	<code>@[entry("_start")]</code>
<code>no_fall</code>	Insert break-like behavior in switch cases.	<code>@[no_fall] switch x;</code>
<code>straight</code>	Use linear switch selection.	<code>@[straight] switch x;</code>
<code>counter</code>	Generate a counted loop.	<code>@[counter(100)] loop</code>
<code>hot/cold</code>	Influence branch layout.	<code>@[cold] if cond;</code>
<code>register</code>	Bind a primitive variable to a register index.	<code>@[register(RAX)] i32 x;</code>
<code>self</code>	Mark a container function whose explicit receiver pointer can be passed through method-like syntax.	<code>@[self] function init(ptr T self)</code>
<code>inline</code>	Influence the function-inlining decision.	<code>@[inline(always)] function f()</code>

```

@[inline(always)]
function bar(); :// Will inline the function under any
                circumstances //

@[inline(never)]
function baz(); :// Will skip this function //

@[inline(model)]
function chloe(); :// Experimental: compact model
                  decides //

```

Listing 11: Inlining annotations.

The plain `@[inline]` form raises the probability that the ordinary inliner will select the function. The `@[inline(always)]` form asks the compiler to inline the function whenever inlining is technically possible. The `@[inline(never)]` form blocks inlining for the annotated function. The experimental `@[inline(model)]` form delegates the choice to a compact neural-network model trained or configured for inlining decisions. This mode is intended for lightweight compiler-internal inference, not for large external model calls, so the decision can remain part of the optimization pipeline.

4 Compiler Architecture

4.1 Pipeline

Figure 1 presents the current CPL compiler architecture as implemented in the repository. The frontend performs preprocessing, tokenization, AST construction, and early semantic checking. The middle-end constructs HIR, converts it to SSA [5], runs SSA-level semantic checking with a Z3-backed symbolic layer [6], and applies HIR-level optimizations. The backend lowers the program to LIR, performs LIR data-flow and copy-propagation passes, selects target-sensitive instructions, allocates registers, applies late peephole cleanup, and finally emits assembly for an external assembler/linker toolchain.

This organization deliberately separates source-level analysis, SSA construction, symbolic checking, high-level optimization, lowering, instruction selection, register allocation, and late cleanup. The separation makes

the implementation suitable not only for code generation, but also for experiments with IR design, pass placement, and target-specific transformations.

4.2 Worked Example

Listing 12 shows a minimal program used to illustrate the compiler forms.

```

start() {
  i32 a = 1;
  i32 b = 2;
  exit a + b;
}

```

Listing 12: Input CPL program.

The corresponding HIR uses stack variables with an `s` suffix and temporaries with a `t` suffix.

```

fn _main0() {
  i32s %0 = alloc;
  i32t %2 = i8n 1 as i32;
  i32s %0 = i32t %2;
  i32s %1 = alloc;
  i32t %3 = i8n 2 as i32;
  i32s %1 = i32t %3;
  i32t %5 = i32s %0 + i32s %1;
  u8t %4 = i32t %5 as u8;
  exit u8t %4;
}

```

Listing 13: High IR (HIR) form for Listing 12.

After lowering to LIR, the program is represented as a basic-block sequence.

```

BB1: start
%2 = $1 as i32;
%0 = %2;
%3 = $2 as i32;
%1 = %3;
%5 = %0 + %1;
%4 = %5 as u8;
exit %4;
BB2: send

```

Listing 14: LIR form before target-sensitive selection.

A later selected LIR form contains target-dependent register choices.

```

BB1: start
rcx = $1;
rcx = rcx;

```

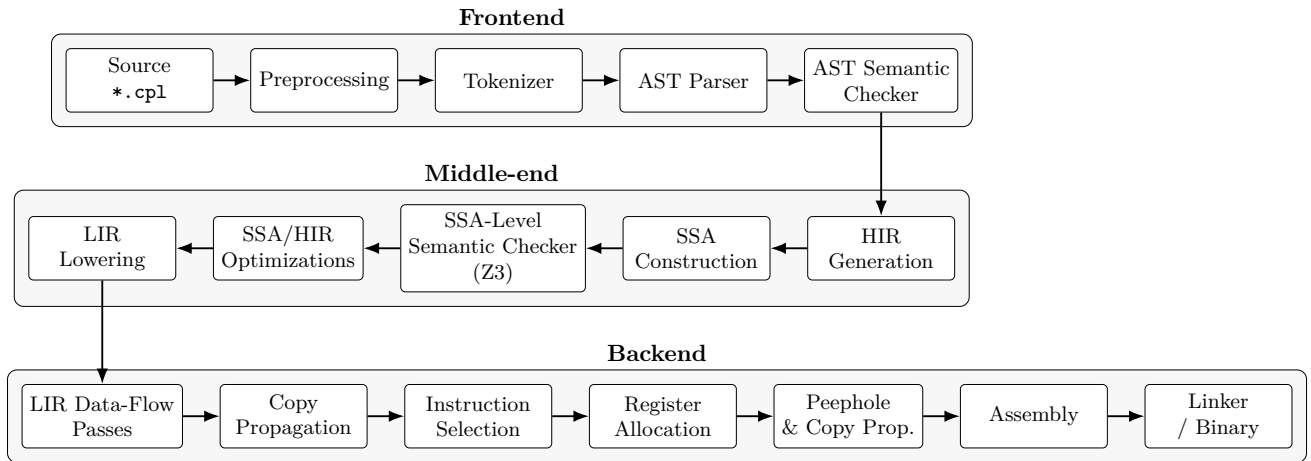


Figure 1: Graphical overview of the CPL compiler architecture. The pipeline is arranged in three phases to preserve readability in a two-column layout while retaining the execution order.

```

rdx = $2;
rdx = rdx;
rax = rcx;
rax = rax + rdx;
rcx = rax;
rcx = rcx;
rdi = rcx;
exit rdi;
BB2: send

```

Listing 15: Selected LIR form.

The final optimized Mach-O assembly for this example collapses the computation to a short sequence.

```

section .text
global _main
_main:
mov al, 1
add al, 2
mov dil, al
mov rax, 0x2000001 ; Exit syscall on MACH064
syscall

```

Listing 16: Generated assembly.

4.3 Backends and Target Status

The implemented backends in this snapshot are x86_64 Mach-O/NASM, x86_64 GNU/Linux NASM, and i386 GNU/Linux NASM. The repository contains selectors, memory selection, caller-saving logic, and assembly generators for these paths, with Mach-O used as the default configuration in the command-line driver. The Linux x86_64 path is present and exercised by parts of the test infrastructure, but it should be treated as less mature than the Mach-O path. The i386 path is a newer target, but it is already useful for freestanding and operating-system-oriented assembly generation tests. Other architecture and system options, such as windows64, should be read as configuration surface rather than publication-ready target support. The compiler can use `ld`, `clang`, or `gcc` as linkers; when external symbols are involved, `clang` or `gcc` are typically more convenient.

4.4 i386 ABI Notes

The i386 backend is intended to interoperate with simple C-style kernel code. Global CPL functions are emitted as externally visible symbols and follow the target’s stack-based calling convention closely enough to be called from C support code when matching prototypes are used. This ABI boundary is deliberately plain: primitive arguments are passed through the caller’s stack frame, return values use the target return register convention, and `extern` declarations describe symbols implemented outside the CPL module.

The `naked` annotation is the main exception to the ordinary function-frame shape. In a normal i386 function, the compiler may establish an `ebp`-based frame. After the usual prologue, `[ebp + 4]` contains the return address, `[ebp + 8]` contains the first argument, and `[ebp + 12]` contains the second argument. In a `naked` function, the generated prologue and epilogue are suppressed. There is no saved base pointer at the top of the frame, so argument access is based on `esp`: the return address is at `[esp]`, the first argument is at `[esp + 4]`, and the second argument is at `[esp + 8]`. This is important for OSDev-style helpers whose stack layout must match the processor’s expectations exactly.

```

@[nosection]
@[naked]
glob function i386_switch2user(u32 eip, u32 esp) ->
i0 {
    asm(eip, esp) {
        "cli",
        "push 0x23",
        "push %1",
        "push 0x202",
        "push 0x1b",
        "push %0",
        "iretd"
    }
}

```

Listing 17: A naked i386 helper whose arguments are addressed from `esp`.

Figure 2 shows the practical effect. The same two source parameters occupy different textual address

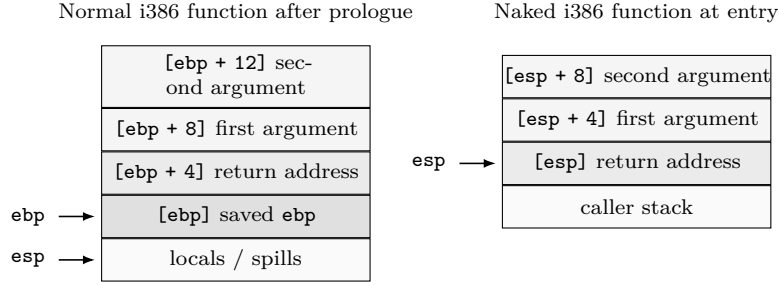


Figure 2: i386 stack view for ordinary and **naked** CPL functions. Without `push ebp; mov ebp, esp`, the backend addresses arguments from `esp` rather than from `ebp`. The **naked** form is useful for OSDev routines such as interrupt-return frame builders, but it makes the programmer responsible for preserving the expected stack shape.

forms depending on whether a frame pointer was created. For the helper in Listing 17, the backend must materialize the CPL parameters from `[esp + 4]` and `[esp + 8]` before expanding the inline assembly placeholders. After user assembly begins to push interrupt-frame words, `esp` changes normally, so hand-written instructions must be written with that movement in mind.

5 Static Diagnostics

CPL contains a two-level diagnostic subsystem. The first level operates on the AST and targets source-structure issues such as read-only variable updates, declaration problems, return mismatches, wrong argument counts, incompatible argument types, unused return values, illegal array access, duplicated branches, invalid function names, dead code, lossy implicit conversions, inefficient infinite loops, invalid `exit` usage, break statements without legal targets, invalid use of `i0`, unused expressions, invalid references, and suspicious alignment requests.

The second level is intentionally placed after SSA construction [5]. At this point the compiler has explicit use-definition chains and phi nodes, so the checker can reason about values and path conditions in a more structured form than in the original HIR. Z3-backed symbolic execution [6] is applied on SSA-form HIR to detect definite null dereferences, null values passed into dereferencing functions, possible null dereferences under path-dependent conditions, and constant branches. This ordering is important: the solver-facing analysis works on SSA-form HIR rather than on pre-SSA HIR, because SSA simplifies symbolic value tracking and makes the generated constraints easier to relate to compiler variables.

The analysis is diagnostic rather than protective. It does not implement memory safety, ownership, or aliasing restrictions, and it does not prevent undefined behavior caused by dangling pointers, unsafe inline assembly, invalid external interfaces, or target-specific misuse.

```
start() {
  function foo(ptr i32 p) -> i32 {
    return dref p; :/ Dereference of NULL /:
  }
  ptr i32 a = 0;
```

```
foo(a);
}
```

Listing 18: Example that triggers SSA-level null-dereference diagnostics.

A representative diagnostic sequence for this example reports both that the return value of `foo(a)` is ignored and that `p` may be dereferenced while equal to null. More complex inputs additionally trigger path-sensitive reports, for example when one branch makes a pointer definitely null while another makes it only conditionally null.

5.1 Z3-Backed Symbolic Layer

The SSA-level checker is implemented as a symbolic layer over the HIR control-flow graph. The analyzer prepares HIR expressions as Z3 formulas, preserves symbolic names for compiler variables, and augments path conditions with phi-node constraints when control-flow edges enter SSA merge points. This design makes it possible to ask local reachability and value questions over the same representation that the optimizer sees.

The current wrapper accepts either parsed JSON or textual HIR dumps, can select a particular function, builds a CFG, and then dispatches solver-backed queries. Two query modes are especially useful for development: `label`, which checks whether a label can be reached under satisfiable path conditions, and `var-eq`, which checks whether a selected variable can, must, or cannot equal a given value. The wrapper also supports initial assignments, pointer-width configuration, strict parsing modes, and cached parsing/analysis artifacts. These features keep the solver path useful not only for compiler diagnostics, but also for investigating IR examples during pass development.

```
python3 z3_wrapper.py input.hir \
  --input-kind dump \
  --function _main \
  --ptr-bits 64 \
  var-eq %7 0
```

Listing 19: Representative Z3-wrapper query shape.

This subsystem should be understood as an experimental symbolic analysis component, not as a formal verification framework. Z3 is used to answer bounded

path and value questions that are helpful for diagnostics such as null-dereference detection and unreachable-code reasoning. It does not by itself provide a complete semantic proof of CPL programs or of compiler transformations [6].

6 Optimization Passes

Table 2 summarizes the implemented optimization passes and how they are enabled by the command-line optimization profiles in the current driver. The driver uses `-O0` as the default, enables LICM, constant optimization, and peephole cleanup at `-O2`, and additionally enables LIR copy propagation and tail-recursion elimination at `-O3`. Function inlining is controlled by an explicit flag rather than by the profile helper, and source annotations can bias or override the ordinary heuristic. CPL also includes an experimental model-guided inlining mode based on compact neural-network models.

6.1 Loop-Invariant Code Motion

LICM operates on SSA High IR. A loop body that repeatedly computes a constant expression such as `10 + 10` can have this computation moved before the loop, with phi nodes preserving loop-carried values.

```
start() {
    i32 d = 0;
    loop {
        i32 c = 10 + 10;
        d += c;
    }
}
```

Listing 20: LICM source pattern.

6.2 Peephole Optimization

Peephole optimization runs late, after instruction selection and register allocation. It removes redundant register-to-itself moves, simplifies decrement-and-branch sequences, and can replace zero materialization with xor-style idioms where applicable.

```
Before:
rax = rcx;
rax = rax - 1;
rcx = rax;
rdx = rcx;
cmp rdx, 0;
je lb11;
jne lb9;

After:
rcx = rcx - 1;
jne lb10;
```

Listing 21: Peephole effect on a counted empty loop.

6.3 PTRN: A Peephole Pattern DSL

The late peephole pass is not only a collection of hand-written rewrites. CPL also uses PTRN, a small domain-specific language for generating peephole optimization

patterns. The motivation is practical: low-level instruction cleanup often consists of many small local rewrites, and encoding them in C by hand makes the pass difficult to extend. PTRN provides a declarative syntax for matching short Low LIR instruction sequences and replacing them with simpler or more efficient sequences.

A PTRN file consists of rewrite rules separated by `/`. The left-hand side describes a sequence to match; the right-hand side after `->` describes the replacement. Pattern objects abstract over concrete Low LIR operands: `areg_N` tracks a repeated abstract register, `aconst_N` tracks a repeated abstract constant, `mem_N` matches memory locations, and `obj` can match an arbitrary object. Conditions such as `[if:equals]`, `[if:arg2:zero]`, or `[if:arg2:mod2]` restrict a match, while actions can transform matched constants in the replacement.

```
; Remove a jump that immediately targets the next
label.
jmp label_1
mklb label_1
->
mklb label_1
/

; Remove redundant self-copy.
mov obj_1, obj_2 [if:equals]
->
delete
/

; Prefer xor-zeroing over moving an immediate zero.
mov areg_1, const 0
->
xor areg_1, areg_1
/
```

Listing 22: Examples of PTRN peephole rules.

The generated C code is then used by the low-level peephole pass. This keeps the compiler backend extensible: adding a new local simplification can be done by adding a pattern rule rather than modifying the peephole engine directly. The approach is intentionally modest; PTRN does not replace global optimization or data-flow analysis. Its role is to make late instruction cleanup explicit, testable, and easier to maintain.

6.4 Function Inlining

The function-inlining pass works on High IR before SSA construction. In its ordinary heuristic mode, it rewards small source functions, penalizes loop-heavy callees, and rewards call sites inside nested loops. The documented heuristic returns true when the score is at least 3. Source annotations can then bias or override this heuristic.

The plain `@[inline]` form increases the odds that a function will be inlined. The `@[inline(always)]` form asks the compiler to inline the function whenever it can technically do so. The `@[inline(never)]` form disables inlining for the function. These annotations affect optimization only; they do not change the source-level meaning of a function.

The experimental `@[inline(model)]` form delegates the inlining choice to compact neural-network models.

Table 2: Optimization passes implemented in the current CPL compiler.

Pass	IR level and role	Level
LICM	Works on SSA HIR; hoists loop-invariant expressions after loop canonicalization.	-02 and above
Peephole	Works on selected LIR after instruction selection, register allocation, memory selection, and caller-saving insertion.	-02 and above
DAG rebuild and sparse constant propagation	Works on HIR via DAG generation and CFG rebuild before lowering to LIR.	-02 and above
Dead-function elimination	Builds and applies a call graph before later HIR transformations.	default pipeline
LIR copy propagation and unused-variable dropping	Rewrites propagated LIR operands and removes unused variable writes.	-03
Tail-recursion elimination	Works on HIR and rewrites tail self-calls to loops before SSA construction.	-03
Function inlining	Works on HIR before SSA using a heuristic over source and destination control-flow properties; may also be guided by <code>inline</code> , <code>inline(always)</code> , <code>inline(never)</code> , or experimental compact neural-model decisions.	explicit flag; known bugs
Hot/cold placement	Implemented during AST-to-HIR generation as annotation-driven branch layout.	default pipeline

The goal is to evaluate whether a small learned policy can improve local inlining choices without making the compiler depend on a large external service or heavy-weight runtime. The model receives compiler-extracted features and returns a decision or score for the inlining pass. The pass exists, but the current implementation has known bugs and should not be presented as fully stable.

7 Examples and Use Cases

The current examples exercise core low-level features rather than large application workloads.

Table 3: Representative CPL programs.

Program	What it demonstrates
Hello World	Strings, pointers to string literals, <code>strlen</code> , inline assembly, direct syscall use, and entry-point termination.
CRC8	Global arrays, indexed memory access, loops, pointer arguments, integer operations, and exit-code calculation.
Brainfuck interpreter	Arrays, argument access, switch dispatch, <code>no_fall</code> , <code>straight</code> , loops, function calls, and byte-level tape manipulation.
Memory and file helpers	Header-style declarations, syscall wrappers, pointer arithmetic, and low-level I/O abstractions.

Listing 23 shows a compact Hello World program using a direct syscall wrapper.

```
function strlen(ptr i8 s) -> i64 {
```

```
    i64 l = 0;
    while dref s; {
        s += 1;
        l += 1;
    }
    return l;
}

function puts(ptr i8 s) -> i0 {
    asm (s, strlen(s)) {
        "push rdi",
        "push rsi",
        "push rdx",
        "mov rax, 33554436",
        "mov rdi, 1",
        "mov rsi, %0",
        "mov rdx, %1",
        "syscall",
        "pop rdx",
        "pop rsi",
        "pop rdi"
    }
}

start(i64 argc, ptr ptr i8 argv) {
    puts(ref "Hello, World!\n");
    exit 0;
}
```

Listing 23: Hello World with explicit syscall wrapper.

7.1 Operating-System Code on i386

A practical use case for CPL is small freestanding routines used by an operating-system kernel. The i386 backend was added to support this style of code: globally visible routines can be emitted as NASM-compatible 32-bit assembly, while `extern` declarations make it possible to integrate CPL modules with C kernel code. In practice, this means that most driver logic can stay in CPL, while the smallest hardware primitives, runtime hooks, and printing functions are supplied by C or by a platform support layer.

Listing 24 is adapted from the repository’s working PS/2 keyboard driver example. The driver uses CPL for state, scancode tables, polling loops, interrupt-handler logic, and exported initialization functions. Hardware-specific operations such as byte I/O and IRQ registration are declared as external functions. Those symbols may be implemented in C, or by C functions that contain the necessary compiler intrinsics or inline assembly. This keeps most of the driver out of raw assembler without hiding the machine boundary.

```
extern function kprintf(ptr i8 fmt, ...) -> i0;
extern function i386_inb(uint16_t port) -> i8;
extern function i386_outb(uint16_t port, u8 data) -> i0;
extern function i386_irq_registerHandler(i32 irq, ptr
    i0 handler) -> i0;

#define KBD_DATA_PORT        0x60
#define KBD_STATUS_PORT      0x64
#define KBD_COMMAND_PORT     0x64
#define KBD_STATUS_OUTPUT_FULL 0x01
#define KBD_STATUS_INPUT_FULL 0x02
#define KBD_CMD_READ_CONFIG  0x20
#define KBD_CMD_WRITE_CONFIG 0x60
#define KBD_CMD_ENABLE_PORT1 0xAE
#define KBD_CONFIG_IRQ1      0x01
#define KBD_CONFIG_PORT1_CLOCK 0x10

glob i8 _curr_char = '\15';
glob arr _key_pressed[128, u8] = { 0 };

function _keyboard_wait_input_clear() -> i8 {
    i32 timeout = 100000;
    while i386_inb(KBD_STATUS_PORT) &
        KBD_STATUS_INPUT_FULL; {
        timeout -= 1;
        if timeout <= 0; return 0;
    }
    return 1;
}

function _keyboard_write_command(u8 command) -> i32 {
    if not _keyboard_wait_input_clear(); return 0;
    i386_outb(KBD_COMMAND_PORT, command);
    return 1;
}

function i386_keyboard_handler(ptr i0 _) -> i0 {
    i8 character = i386_inb(KBD_DATA_PORT);
    if character < 0 || character >= 128; return;

    _key_pressed[character as i32] = 0;
    if not (character & 0x80); {
        _key_pressed[character as i32] = 1;
        _curr_char = _alphabet[character as i32];
    }
}

glob function i386_init_keyboard() -> i0 {
    u8 config = 0;
    if _keyboard_write_command(KBD_CMD_READ_CONFIG);
        _keyboard_read_data(ref config);

    config |= KBD_CONFIG_IRQ1;
    config &= 0xFF ^ KBD_CONFIG_PORT1_CLOCK;
    if _keyboard_write_command(KBD_CMD_WRITE_CONFIG);
        _keyboard_write_data(config);

    _keyboard_write_command(KBD_CMD_ENABLE_PORT1);
    i386_irq_registerHandler(1, i386_keyboard_handler
    );
}
```

Listing 24: Excerpt from an i386 PS/2 keyboard driver using C externs.

From the C side, the imported and exported symbols are ordinary low-level ABI boundaries. For example, the platform layer can provide byte-port primitives and the kernel can call the CPL initialization routine through matching prototypes:

```
uint8_t i386_inb(uint16_t port);
void i386_outb(uint16_t port, uint8_t data);
void i386_irq_registerHandler(int irq, void (*handler)
    )(void*);

void i386_init_keyboard(void);
```

Listing 25: C boundary for the CPL keyboard driver.

This is the preferred shape for larger system code. CPL can express the driver’s control flow and data structures, the compiler can still perform parsing, HIR construction, diagnostics, and target-specific regression tests, and the C boundary isolates the operations that are genuinely platform-specific. Inline `asm` remains useful for tiny routines that must be exactly a particular instruction sequence, such as loading the task-state segment or constructing an interrupt-return frame, but it is not required for every hardware-facing driver.

8 Testing Infrastructure

The compiler is tested with a phase-oriented framework rather than only through end-to-end examples. The core idea is to make each compiler stage observable: a test can stop after preprocessing, tokenization, AST generation, semantic analysis, HIR construction, SSA construction, DAG construction, constant-folding analysis, LIR construction, instruction selection, instruction planning, register allocation, peephole optimization, assembly generation, or assembly-level constant folding. This makes regressions easier to localize than in a purely black-box compiler test.

8.1 Integrated Pipeline Testing

The integration tester is built around a dedicated test driver that exercises the full compilation pipeline while allowing individual sections to be enabled or disabled at compile time. The active section is selected by a module argument. A typical invocation is:

```
python3 integrated_testing.py --run --module <name> \
    --debugger <lldb/gdb> \
    --test-code <path> \
    --extra-flags <flag>
```

Listing 26: Integrated testing command.

The current module set covers the main compiler phases:

For example, a basic assembly-generation regression test can be run by selecting the `asm` module. Memory-manager diagnostics can also be enabled through an extra compile-time flag and then post-processed by a leak analysis script.

```
python3 integrated_testing.py --run --module asm

python3 integrated_testing.py --run --module hir_dag
\
```

Table 4: Compiler phases exposed by the integration testing framework.

Frontend	HIR / SSA	LIR / backend	Assembly
preproc	hir	lir	asm
prep	hir_ssa	lir_constfold	asm_constfold
ast	hir_dag	lir_selector	
sem	hir_constfold	lir_instplan	
		lir_regalloc	
		lir_peekhole	

```
--test-code dummy_data/test2.cpl \
--extra-flags DMEM_OPERATION_LOGS
python3 leaks.py --log-path <path_to_log>
```

Listing 27: Assembly and leak-oriented test invocations.

8.2 Module Tests

In addition to integration testing, the compiler has a module-level unit testing mode. A module test directory contains a C test harness and a dependency description. The harness computes the expected result for the selected compiler component, while the dependency file describes which compiler modules must be linked for the test. This mode is invoked as follows:

```
python3 module_testing.py --path <path>
```

Listing 28: Module testing command.

This separation is useful for testing compiler internals whose output is not naturally observable through a final executable. For example, AST construction, ownership-like metadata, data-flow sets, or register-allocation artifacts may be compared against expected textual output.

8.3 Test Oracle Format

CPL test files include an `OUTPUT` block that defines the expected compiler or runtime output. The framework also supports test flags placed at the beginning of a file. The most important flags are: `: RUN_ASM :`, which builds and executes the generated assembly; `: RUN_ASM[args=...] :`, which executes the same test with several argument sets; `: BLOCK_TEST :`, which makes a failing test fail the whole run; `: BUG :`, which records an expected failing test; `: TEST_DEBUG :`, which runs under `gdb` or `lldb`; and `: LEAK_TRACE :`, which enables memory logging for leak localization.

The oracle language is intentionally tolerant where compiler dumps contain unstable identifiers. The marker `{X}` accepts an arbitrary substring, `{content}` requires a line to occur somewhere in the output, and `<content>` checks that a line contains a space-insensitive sequence. Runtime tests can also assert exit codes and select per-argument expected outputs through `@exit_code` and `@case_index`.

```
: RUN_ASM[args="apple"|args="banana"] :
function putc(i8 c) -> i0 {
    syscall(0x2000004, 1, ref c, 1);
}
@[entry("_main")]
```

```
function main(i32 argc, ptr ptr i8 argv) -> i0 {
    putc(argv[1][0]);
    exit 0;
}

:/ OUTPUT
@case_index=0
a
---
@case_index=1
b
---
/:
```

Listing 29: Runtime test with multiple argument cases.

8.4 Representative Regression Programs

The supplied regression programs exercise both language features and backend behavior. They are also useful as benchmark kernels because the same `OUTPUT` blocks can request repeated measurements and generated-assembly line counts.

This testing setup does not amount to a proof of correctness. It is nevertheless important for a multi-stage compiler because it checks both internal forms and executable behavior, supports expected-failure tracking, and provides a practical way to detect regressions in individual passes before they are hidden by later lowering stages.

9 Preliminary Evaluation

The current evaluation should be read as a preliminary microbenchmark study rather than a broad performance claim. The chosen kernels are intentionally small and expose loop overhead, arithmetic simplification, and pointer-heavy traversal. They are useful for inspecting generated code and optimization effects, but they are not a substitute for large application benchmarks.

All available timings were collected on macOS Catalina 10.15.7 using an Intel i7-3650QM CPU at 2.4 GHz and 16 GB DDR3 memory. The compared tools were GCC 14.2.0, Apple Clang 12.0.0, a Mach-O Cordell compiler build, and rustc 1.87.0. GCC and Clang were tested at `-O0` and `-O3`, CPL at `-O0` and `-O3`, and Rust at `opt-level=0` and `opt-level=2`. Each timing was gathered five times after compilation. The measurements use different timing wrappers for CPL and for the C-family baselines, so the numbers should be interpreted conservatively. In particular, the empty-loop

Table 5: Representative CPL regression programs used by the current testing corpus.

Test program	Purpose
01_simple_print.cpl	String allocation, <code>strlen</code> , system call output, runtime output matching.
02_count_to_billion.cpl	Counted loop lowering, <code>naked</code> entry, exit-code checking, loop-overhead measurement.
03_crc8.cpl	Global lookup tables, pointer arguments, string scanning, indexed memory access, multi-argument runtime cases.
04_arith_mix.cpl–	Arithmetic kernels, branches, function-call overhead, hot-loop behavior, exit-code validation.
06_function_call_hot.cpl	Table traversal, pointer/string scan, loop-carried values, integer overflow behavior, generated-code-size measurements.
07_table_sum_hot.cpl–	Mixed arithmetic and function-call stress tests.
09_fibonacci.cpl	
10_second_arith_mix.cpl–	
11_third_arith_mix.cpl	
12_brainfuck.cpl	Larger interpreter-style program using arrays, switch dispatch, <code>no_fall</code> , <code>straight</code> , pointer movement, and output checking.
13_simple_switch.cpl	Small switch-dispatch regression with command-line arguments and exit-code assertion.

benchmark should be understood as a loop-overhead test: the C baseline uses `asm volatile` to prevent the compiler from deleting the loop.

9.1 Benchmarked Kernels

Listing 30 shows the counted empty loop used to evaluate loop overhead and late low-level simplification.

```
@[naked] start() {
    @[counter(1000000000)] loop {
    }
    exit 0;
}
```

Listing 30: Empty-loop benchmark kernel.

Listing 31 shows the Fibonacci kernel used to exercise simple arithmetic and loop-carried values.

```
start() {
    i32 a = 1;
    i32 b = 0;
    @[counter(1000000)] loop {
        i32 tmp = a;
        a = a + b;
        b = tmp;
    }
    exit b;
}
```

Listing 31: Fibonacci benchmark kernel.

Listing 32 shows the string-traversal kernel, which is more representative of pointer-based code generation.

```
start() {
    str msg = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789+/-";
    i64 outer = 0;
    u8 acc = 0;
    ptr i8 p = ref msg;

    while outer < 1000000; {
        p = ref msg;
        while dref p; {
            acc = (acc + dref p) & 0xFF;
            p += 1;
        }
        outer += 1;
    }

    exit acc;
}
```

Listing 32: String-traversal benchmark kernel.

9.2 Optimized Runtimes

Table 6 reports the optimized runtimes. Figure 3 visualizes the same measurements.

The results show that optimized CPL code can produce compact and reasonably fast machine code for these small kernels. They should not be generalized to whole-program performance. The string-traversal kernel remains slower than Clang, GCC, and Rust, which indicates that pointer-heavy workloads are a more demanding target for the current backend.

9.3 Effect of Optimization within CPL

Figure 4 shows the effect of CPL optimization alone by comparing `-O0` and `-O3`. The empty loop benefits the most, which is consistent with the late peephole simplifications described earlier.

The optimized CPL assembly sizes reported for these kernels are compact: 11 lines for the empty loop, 17 lines for Fibonacci, and 84 lines for string traversal. These values are not a full proxy for performance, but they do suggest that the backend is capable of collapsing simple kernels to short instruction sequences.

10 Limitations and Threats to Validity

The current report should be read with the following limitations in mind.

No formal semantics or proof. CPL does not currently provide a mechanized semantics, compiler-correctness proof, or translation validation.

No memory safety. The static analyzer can find selected problems, but CPL intentionally does not implement an ownership or borrowing discipline.

Limited target validation. The most exercised path is x86_64 Mach-O/NASM. x86_64 GNU/Linux NASM is implemented but less tested. The i386 GNU/Linux NASM backend has dedicated assembly-generation tests and real operating-system helper examples, but it is newer and has not yet been evaluated with the same breadth as the Mach-O path. Other architecture and system-type options exposed by the

Table 6: Optimized benchmark runtimes in seconds. Lower is better.

Benchmark	CPL -O3	Clang -O3	GCC -O3	Rust opt-level=2
Empty loop	0.702	0.748	0.758	0.333
Fibonacci	0.412	0.417	0.412	0.335
String traversal	0.513	0.430	0.470	0.332

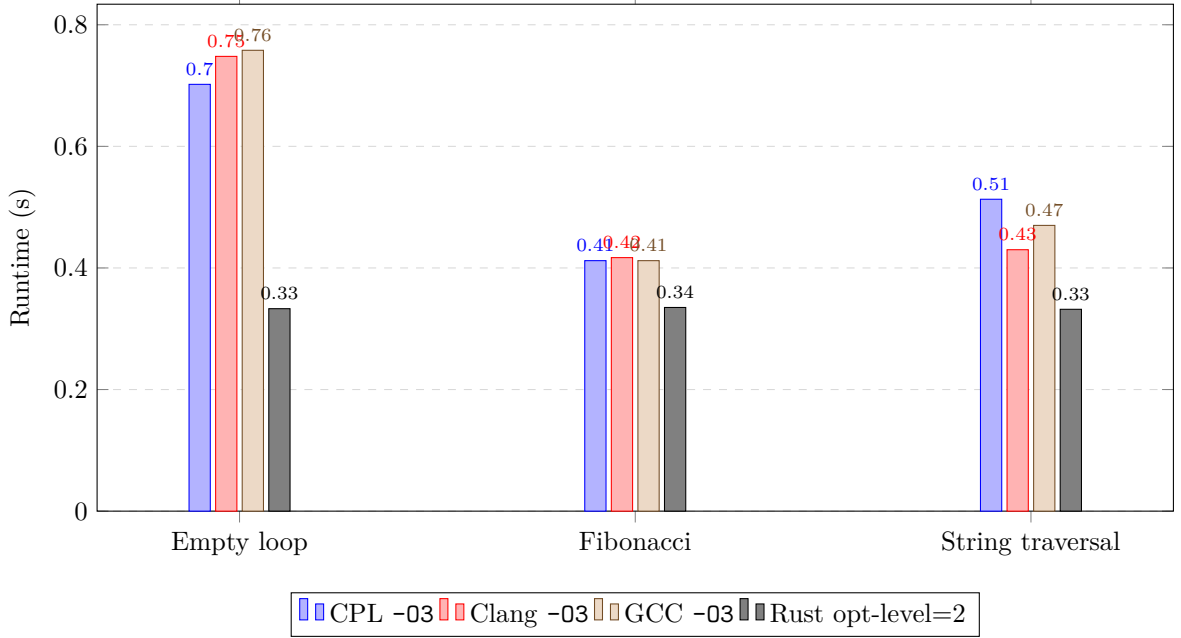


Figure 3: Optimized runtimes for the three benchmark kernels.

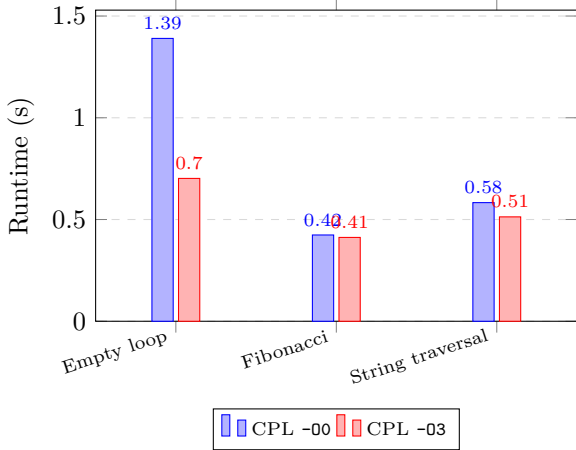


Figure 4: Optimization effect within CPL.

driver should be treated as planned or partial support unless backed by dedicated tests.

Known function-inlining bugs. Function inlining exists, including annotation-driven and experimental model-guided modes, but it is not stable enough to be used as an unconditional compiler-quality claim. The compact neural-model mode should be treated as an experiment rather than as a validated optimization policy.

Small benchmark suite. The benchmark suite consists of microbenchmarks and small interpreters. It does not include large applications, compiler self-hosting, broad target comparisons, or Linux x86_64 replication.

Evaluation snapshot. The benchmark data was collected for a particular Mach-O compiler snapshot and hardware configuration. It should therefore be treated as preliminary evidence for the implementation direction rather than as a complete evaluation of every backend and optimization combination.

Inline assembly opacity. Inline assembly is copied into the output after placeholder substitution and is not optimized by the compiler. This can invalidate assumptions made by surrounding optimization passes if the programmer uses labels, jumps, or unpreserved registers in fragile ways.

11 Related Work

CPL is best understood as a small systems-language and compiler-construction testbed, not as a verified or production language.

Compiler frameworks. LLVM is a modular compiler infrastructure designed to support program analysis and transformation over a common low-level representation [10]. CPL is much smaller and does not

attempt to provide a general reusable ecosystem, but it shares the idea that intermediate representations are central to optimization and backend experimentation.

SSA and optimization. CPL’s SSA stage connects to the long-standing use of SSA as a representation for data-flow optimization [5]. The current implementation uses SSA as one stage in a larger HIR-to-LIR pipeline rather than as a complete formal IR specification. The symbolic analysis layer uses Z3, an SMT solver widely used for program analysis and verification-oriented tooling [6].

Verified systems languages and compilers. Coq presents a restricted systems language with a self-certifying compiler and Isabelle/HOL artifacts [13]. Low* embeds verified low-level programming in F* and provides memory safety and verification support [14]. Jasmin targets efficient and formally verified cryptographic implementations [1]. CompCert is a formally verified C compiler with machine-checked semantic preservation guarantees [11, 19]. CPL differs from all of these: it does not claim verified compilation, formal semantics, memory safety, or cryptographic security preservation.

Implementation reports for smaller languages. The Dino implementation report is close in genre: it documents language implementation decisions, VM/JIT design, and benchmark-driven implementation experience [12]. The improved GP 2 compiler work shows how implementation changes to an experimental language compiler can yield substantial performance improvements [4]. GNU epsilon provides a different example of a minimalistic language core intended for extensibility and analysis [16]. These works motivate reporting not only language syntax, but also implementation architecture and measured effects.

Educational compilers. Petit and small self-contained compiler courses emphasize educational compiler construction, staged language growth, and transparent implementation [2, 3]. CPL has an educational motivation, but differs by exposing system-level constructs such as syscalls, inline assembly, low-level annotations, and explicit backend concerns.

Compiler testing. Csmith demonstrates the effectiveness of randomized differential testing for finding C compiler bugs [20]. CPL currently does not include a comparable fuzzing methodology. Such testing is an important direction for future work, especially because the compiler contains multiple lowering and optimization stages.

12 Future Work

The most important next steps are:

- strengthen Linux x86_64 target validation;

- broaden i386 validation beyond assembly-generation tests and kernel helper routines;
- repair known function-inlining bugs and validate the compact neural-model-guided inline mode;
- publish reproducible benchmark scripts and raw data;
- extend the existing phase-oriented test suite with differential and randomized testing inspired by compiler-fuzzing work;
- document the IR formats more formally;
- improve diagnostics for inline assembly boundaries and target-specific calls;
- extend the PTRN rule set and add rule-level regression tests for peephole rewrites;
- expand the Z3 layer from ad-hoc investigations toward a more systematic SSA-level analysis API;
- expand the set of real programs beyond microbenchmarks and small interpreters.

Classes, enums, constructors, destructors, inheritance, and ownership-oriented aggregate models remain outside the current core by design. Containers provide a deliberately small C-like aggregate mechanism, but richer user-defined type systems may be studied separately. This report treats that restriction as part of CPL’s deliberate language model rather than as an accidental omission.

13 Conclusion

CPL is a compact low-level language and compiler infrastructure for studying systems-language design and compiler implementation. It combines explicit low-level constructs, lightweight containers, a restricted syntax, a multi-stage compiler pipeline, static diagnostics, SSA-based optimization, annotation-guided and experimental model-guided inlining, low-level backend passes, and target-specific assembly generation. Its value lies in being small enough to inspect and modify while still exposing problems that arise in real compilers: lowering boundaries, register allocation, branch layout, null reasoning, inline assembly opacity, and benchmark interpretation. The current implementation is not formally verified, memory-safe, production-ready, or self-hosting. Within those limits, CPL provides a useful experimental platform for compiler architecture and systems-programming research.

References

- [1] Santiago Arranz-Olmos, Gilles Barthe, Lionel Blatter, Benjamin Grégoire, Vincent Laporte, and Paolo Torrini. The Jasmin compiler preserves cryptographic security, 2025.

- [2] Raul Brajczewski Barbosa. Petit programming language and compiler, 2023.
- [3] Daniil Berezun and Dmitry Boulytchev. Reimplementing the wheel: Teaching compilers with a small self-contained one, 2022.
- [4] Graham Campbell, Jack Romo, and Detlef Plump. The improved GP 2 compiler, 2020.
- [5] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, 13(4):451–490, 1991.
- [6] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008.
- [7] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages*, 2(POPL):66:1–66:34, 2018.
- [8] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 2 edition, 1988.
- [9] Steve Klabnik, Carol Nichols, and Chris Krycho. *The Rust Programming Language*. No Starch Press, 2 edition, 2023.
- [10] Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis and transformation. In *Proceedings of the International Symposium on Code Generation and Optimization*, pages 75–88, 2004.
- [11] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.
- [12] Vladimir N. Makarov. Implementation of the programming language Dino – a case study in dynamic language performance, 2016.
- [13] Liam O’Connor, Christine Rizkallah, Zilin Chen, Sidney Amani, Japheth Lim, Yutaka Nagashima, Thomas Sewell, Alex Hixon, Gabriele Keller, Toby Murray, and Gerwin Klein. COGENT: Certified compilation for a functional systems language, 2016.
- [14] Jonathan Protzenko, Jean-Karim Zinzindohoué, Aseem Rastogi, Tahina Ramananandro, Peng Wang, Santiago Zanella-Béguelin, Antoine Delignat-Lavaud, Cătălin Hrițcu, Karthikeyan Bhargavan, Cédric Fournet, and Nikhil Swamy. Verified low-level programming embedded in F*. *Proceedings of the ACM on Programming Languages*, 1(ICFP):17:1–17:29, 2017.
- [15] D. M. Ritchie, S. C. Johnson, M. E. Lesk, and B. W. Kernighan. UNIX time-sharing system: The C programming language. *The Bell System Technical Journal*, 57(6):1991–2019, 1978.
- [16] Luca Saiu. GNU epsilon – an extensible programming language, 2012.
- [17] Bjarne Stroustrup. Evolving a language in and for the real world: C++ 1991–2006. In *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages*, HOPL III, pages 4–1–4–59. ACM, 2007.
- [18] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley, 4 edition, 2013.
- [19] The CompCert Project. *The CompCert Verified Compiler*, 2026. Accessed 2026-05-12.
- [20] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 283–294. ACM, 2011.
- [21] Zig Software Foundation. *The Zig Programming Language Reference*, 2026. Accessed 2026-05-12.